

基于拓扑排序的课程先修关系分析

系统设计

学院：计算机科学与技术学院

专业：软件工程

班级：42001902

学号：2452654

姓名：郭炫君

老师：唐剑锋

完成日期：2025.12.3

1 引言

在高等教育的课程体系中，学生每学期都面临着如何科学、合理地规划课程选择的难题。许多专业课程之间存在严格的先修与后修关系，形成一个复杂的知识依赖网络。对于学生而言，手动梳理这些依赖关系，安排出既满足所有课程先决条件、又符合个人学习进度的课表，不仅耗时费力，而且极易出错。因此，开发一个能够自动分析课程依赖关系、辅助学生制定修读计划的工具，具有迫切的现实需求。

这一现实问题，恰好可以通过离散数学中关于图论和拓扑排序的核心理论得到系统性的解决。离散数学为这类依赖关系的表示与分析提供了严谨的数学模型和高效的算法框架。具体而言，有向无环图（Directed Acyclic Graph, DAG）和拓扑排序（Topological Sorting）等概念，正是刻画任务间偏序关系、求解线性执行顺序的关键理论工具。借助这些理论，我们不仅可以从数学上严格判断一个课程体系是否存在逻辑上的循环依赖（即图中是否存在环），还能在确保无环的前提下，生成所有满足依赖关系的、合法的课程修读顺序。

基于此，本项目旨在设计并实现一个基于拓扑排序的课程先修关系分析系统。该系统将帮助用户完成依赖图建模、环路检测及所有合法修课顺序的枚举。本系统的开发，一方面验证了离散数学理论在解决实际问题中的强大能力，另一方面也为学生和教务管理者提供了一个直观、可靠、高效的课程规划辅助工具。

2 问题描述

课程先修关系分析问题的核心在于对课程及其依赖关系进行数学建模。系统需要处理的具体问题可以形式化描述为：给定一个课程集合 $C = \{C_1, C_2, \dots, C_n\}$ ，其中每个课程 C_i 由一个字符串标识；同时给定一个先修关系集合 $R = \{(C_i, C_j) \mid C_i \text{ 是 } C_j \text{ 的先修课}\}$ ，表示课程之间的依赖关系。这些关系构成了一个有向图 $G = (C, R)$ ，其中节点代表课程，有向边代表先修关系。

系统的核心任务包括两个方面：首先，需要判断图 G 中是否存在环。如果存在环，则意味着存在循环依赖，例如课程 A 是 B 的先修课， B 是 C 的先修课，而 C 又是 A 的先修课，这样的依赖关系无法在实际排课过程中实现。其次，如果图 G 是无环的（即构成 DAG），则需要找出所有可能的拓扑排序序列。拓扑排序序列是一个课程序列 $[C_1, C_2, \dots, C_n]$ ，满足对于任意先修关系 (C_i, C_j) ，课程 C_i 在序列中都出现在 C_j 之前。

这个问题的解决不仅需要处理课程数量较多时的计算效率问题，还需要考虑内存使用的限制。在实际应用中，课程数量可能达到数十门，先修关系可能达到数百条，因此算法的设计和实现必须充分考虑效率和资源的合理使用。

3 系统设计概述

系统设计的总体思路遵循软件工程的模块化原则，将复杂问题分解为多个相对独立的子问题，分别设计解决方案，然后整合为一个完整的系统。下图是本次系统设计的系统框架图。

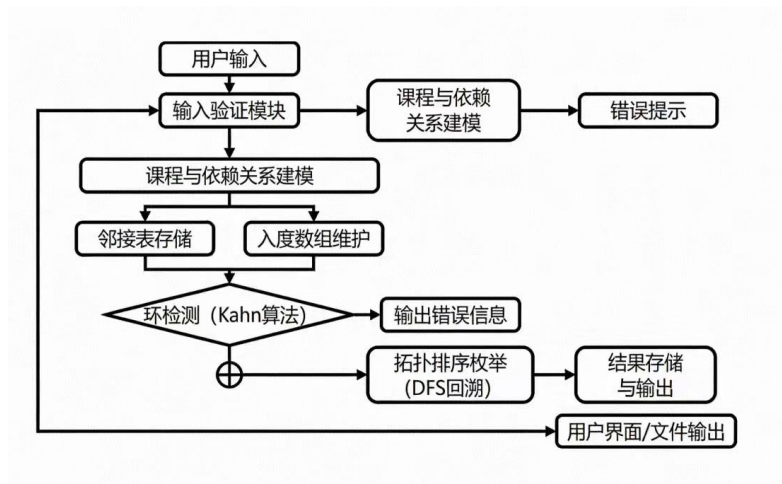


图 1 系统框架图

如图 1 所示，系统框架图清晰地展示了从用户输入到结果输出的完整处理流程，涵盖了输入验证、关系建模、核心算法执行与结果输出四大阶段。各模块之间通过数据流紧密连接，形成一个层次分明、逻辑清晰的系统结构。接下来将阐述各部分的设计思路。

3.1 课程存储与映射

由于课程名称是字符串形式，而算法操作通常需要整数标识符，因此需要建立课程名称到整数索引的双向映射关系。使用二维字符数组存储课程名称，并通过线性查找实现名称到唯一编号的映射。

3.2 依赖关系建模

使用邻接表存储课程的关系。与邻接矩阵相比，邻接表在稀疏图的情况下具有明显的空间优势，而且能够高效地遍历每个节点的后继节点。同时维护每个课程的入度，以支持高效的图遍历与入度查询。

3.3 环检测机制

在生成拓扑序列前，使用 Kahn 算法判断图中是否存在环，确保拓扑排序的可行性。

3.4 拓扑序列枚举

在确保图无环的基础上，系统需要枚举所有可能的拓扑排序序列。通过回溯（DFS）方法，动态维护入度与已选课程状态，枚举所有合法的拓扑序列，并通过常数限制避免内存溢出。

4 数据结构设计

系统采用了多种数据结构来支持不同功能模块的需求，这些数据结构的选择充分考虑了算法的需求和实现的效率。

4.1 课程存储

课程名称存储采用了二维字符数组 `nameList[NMAX][NAMELEN]`，其中 `NMAX` 定义了最大课程数量（50），`NAMELEN` 定义了课程名称的最大长度（15 个字符，

包括结尾的空字符)。为了支持课程名称到索引的快速查找，系统实现了 `find_course` 函数，采用线性搜索的方式在 `nameList` 中查找指定课程名称。虽然线性搜索的时间复杂度为 $O(n)$ ，但考虑到实际课程数量不会太大，这种简单实现是合理的选择。

4.2 邻接表结构

图的存储采用了邻接表结构，由三个数组共同实现：`head[NMAX]`存储每个课程节点对应的第一条边的索引；`to[EMAX]`存储每条边指向的目标节点；`nextEdge[EMAX]`存储下一条同起点的边。在添加边时，系统使用头插法将新边插入邻接表。由于拓扑排序不关心边的遍历顺序，这种实现方式是完全可以接受的。

4.3 记录节点状态的数组

为了支持拓扑排序算法的实现，系统维护了多个状态数组。`indegree[NMAX]`记录了每个课程的初始入度，用于检测环和初始化拓扑排序；`degCopy[NMAX]`是 `indegree` 的副本，在 DFS 回溯过程中使用，避免修改原始入度信息；`used[NMAX]`是布尔数组，标记课程是否已被选入当前路径；`path[NMAX]`记录当前的拓扑序列。这些数组共同维护了算法的状态，确保了回溯的正确性。

4.4 结果存储

结果存储采用了二维数组 `allTopo[MAX_ORDER][NMAX]`，其中 `MAX_ORDER` 定义了最大拓扑序列数量（1000）。这个限制是为了防止在课程数量较多、拓扑序列数量指数增长时导致内存溢出。虽然在理论上可能会丢失部分结果，但在实际应用中，1000 个方案已经足够为学生提供全面的参考。

5 核心算法实现

系统中采用了两种核心算法，下面的表格分析了其复杂度特征，理解这些复杂度特征有助于评估算法在实际应用中的性能表现，并为系统优化提供理论依据。

算法名称	时间复杂度	空间复杂度	主要功能
Kahn 环检测算法	$O(V+E)$	$O(V)$	检测图中是否存在环
DFS 回溯枚举算法	$O(V!*E)$ (最坏情况)	$O(V+E+P*V)$	枚举所有拓扑排序

表 1 核心算法复杂度表格

注：V 为课程数量，E 为先修关系数量，P 为输出的拓扑序列数量

由表 1 可见，Kahn 算法的时间复杂度与图规模成线性关系，适合在课程数量较多时快速完成环检测。相比之下，DFS 回溯算法在最坏情况下具有阶乘级复杂度，但其能够系统性地枚举所有合法拓扑序列。

5.1 kahn 算法环检测

在课程先修关系中，如果存在环，即某门课程直接或间接成为自己的先修课

程，那么拓扑排序将不存在，也就无法生成合法的修课顺序。为此，程序首先采用经典的 Kahn 算法对输入构建的先修关系图进行环检测。该算法通过不断移除入度为 0 的节点来判断图中是否存在环。具体实现中，程序首先创建原始入度数组的副本 `degCopy`，然后在循环中不断查找入度为 0 且未被访问的课程节点。一旦找到这样的节点，程序就将其标记为已访问，即在 `used` 数组中标记为 1，并将该节点所有后继课程的入度减 1，以反映该节点被移除对图结构的影响。如果在某次循环中无法找到入度为 0 的节点，但仍有课程未被访问，则说明图中存在环路，程序据此判断输入数据不合法。通过这种方式，Kahn 算法能够高效地检测有向图中的环路，为后续拓扑排序的生成提供前提条件，并保证生成的修课顺序符合离散数学中拓扑排序的定义。

5.2 回溯法枚举拓扑排序

在确认课程先修关系合法后，程序采用深度优先搜索结合回溯的方法枚举图的所有拓扑排序，从而生成所有可能的合法修课顺序。与生成单个拓扑排序不同，枚举所有拓扑排序需要探索每一条可能的选择路径。程序核心函数 `dfs_all` 通过递归实现这一过程。在每一层递归中，程序首先遍历所有课程，收集当前入度为 0 且未被访问的课程存放到 `candidate` 数组作为候选，然后对每个候选课程依次进行尝试。对于某个候选课程 `u`，程序将其加入当前路径并标记为已访问，同时将课程 `u` 的所有后继课程入度减 1，以反映该选择对后续节点的影响。随后递归进入下一层，尝试填充路径的下一个课程。递归返回后，为保证各条搜索路径互不干扰，程序会撤销当前选择，将候选课程 `u` 标记为未访问，并恢复 `u` 的后继课程的入度状态，从而确保状态恢复到选择前的状态。只有当当前路径长度等于课程总数时，程序才会将该完整拓扑序列记录到全局结果数组中。通过这种深度优先搜索与回溯结合的策略，程序能够完整枚举所有合法的修课顺序，同时保证每条路径的独立性和正确性，为课程安排提供全面而可靠的依据。

6 系统实现细节

系统的实现过程中，除了核心算法外，还包含了许多重要的细节处理，这些细节直接影响了系统的健壮性、可用性和用户体验。

6.1 输入处理

输入处理是系统的重要组成部分。系统对输入进行严格验证：如果输入无效，系统会提示错误信息并要求重新输入，直到获得有效输入为止。这种设计避免了因用户误操作导致的程序崩溃或错误结果。

- (1) 课程数量必须在 2 到 `NMAX` 之间，并且不能非法输入。
- (2) 输入的课程名称不能重复。
- (3) 先修关系数量必须在 1 到 `NMAX` 之间，并且不能非法输入。
- (4) 输入的先修关系不能输入不存在的课程，不能输入自己是自己的先修课程，不能输入重复的先修关系。

6.2 输出设计

输出设计方面，系统采用了清晰易读的格式。每个拓扑序列以"方案 `x`: 课程 `A` -> 课程 `B` -> ..."的形式输出，箭头清楚地表示了课程的先后顺序；在输入先修关系的时候，会提示当前是第几个先修关系。

6.3 防御性编程的设计

在 `add_course` 函数中，除了检查课程数量是否超过 `NMAX` 限制外，还使用 `strncpy` 函数而不是 `strcpy` 函数来复制课程名称，确保不会发生缓冲区溢出；在 `add_edge` 函数中，检查边数是否超过 `EMAX` 限制；在 `dfs` 递归获取拓扑排序时会检查 `totalTopo` 是否超过 `MAX_ORDER`。这些安全检查极大地提高了系统的健壮性。

7 测试与验证

7.1 错误输入处理功能展示

```
请输入课程数量: qw
课程数量必须是整数，请重新输入！
请输入课程数量: 1
课程数量超出范围，应该在2到50之间！
请输入课程数量: 5
请输入 5 个课程名称（每个不超过 19 个字符，用空格或换行分隔）：
数据结构 数据结构
请不要输入同名的课程！
离散数学 计算机组成原理 高等数学 线性代数
请输入先修关系数量: a
先修关系数量必须是整数，请重新输入！
请输入先修关系数量: 0
先修关系数量超出范围，应该在1到200之间！
请输入先修关系数量: 3
请输入每条先修关系（格式A B 表示A是B的先修课）
1: 数学分析 高等数学
输入的课程不存在，请重新输入！
1: 高等数学 高等数学
课程不能作为自己的先修课，请重新输入！
1: 高等数学 离散数学
2: 高等数学 离散数学
该先修关系已经存在，请重新输入！
2: _
```

7.2 存在环关系情况

```
请输入课程数量: 4
请输入 4 个课程名称（每个不超过 19 个字符，用空格或换行分隔）：
数据结构 离散数学 概率论 操作系统
请输入先修关系数量: 4
请输入每条先修关系（格式A B 表示A是B的先修课）
1: 数据结构 离散数学
2: 离散数学 概率论
3: 概率论 数据结构
4: 操作系统 数据结构

该课程依赖图中存在环！没有任何拓扑序（课程顺序无解）。
```

7.3 正常功能测试


```
请输入课程数量: 5
请输入 5 个课程名称 (每个不超过 19 个字符, 用空格或换行分隔):
高等数学 线性代数 概率论 数据结构 算法设计
请输入先修关系数量: 4
请输入每条先修关系 (格式A B 表示A是B的先修课)
1: 高等数学 概率论
2: 线性代数 概率论
3: 概率论 数据结构
4: 数据结构 算法设计

共找到 2 个合法的修课顺序:

方案 1: 高等数学 -> 线性代数 -> 概率论 -> 数据结构 -> 算法设计
方案 2: 线性代数 -> 高等数学 -> 概率论 -> 数据结构 -> 算法设计
```

```
请输入课程数量: 8
请输入 8 个课程名称 (每个不超过 19 个字符, 用空格或换行分隔):
高等数学 线性代数 离散数学 概率论 数据结构 算法设计 操作系统 计算机网络
请输入先修关系数量: 7
请输入每条先修关系 (格式A B 表示A是B的先修课)
1: 高等数学 概率论
2: 线性代数 概率论
3: 离散数学 数据结构
4: 概率论 数据结构
5: 数据结构 算法设计
6: 数据结构 操作系统
7: 算法设计 计算机网络

共找到 24 个合法的修课顺序:

方案 1: 高等数学 -> 线性代数 -> 离散数学 -> 概率论 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 2: 高等数学 -> 线性代数 -> 离散数学 -> 概率论 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 3: 高等数学 -> 线性代数 -> 离散数学 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 4: 高等数学 -> 线性代数 -> 概率论 -> 离散数学 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 5: 高等数学 -> 线性代数 -> 概率论 -> 离散数学 -> 数据结构 -> 算法设计 -> 计算机网络 -> 操作系统
方案 6: 高等数学 -> 线性代数 -> 概率论 -> 离散数学 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 7: 高等数学 -> 离散数学 -> 线性代数 -> 概率论 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 8: 高等数学 -> 离散数学 -> 线性代数 -> 概率论 -> 数据结构 -> 算法设计 -> 计算机网络 -> 操作系统
方案 9: 高等数学 -> 离散数学 -> 线性代数 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 10: 线性代数 -> 高等数学 -> 离散数学 -> 概率论 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 11: 线性代数 -> 高等数学 -> 离散数学 -> 概率论 -> 数据结构 -> 算法设计 -> 计算机网络 -> 操作系统
方案 12: 线性代数 -> 高等数学 -> 离散数学 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 13: 线性代数 -> 高等数学 -> 离散数学 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 14: 线性代数 -> 高等数学 -> 概率论 -> 离散数学 -> 数据结构 -> 算法设计 -> 计算机网络 -> 操作系统
方案 15: 线性代数 -> 高等数学 -> 概率论 -> 离散数学 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 16: 线性代数 -> 离散数学 -> 高等数学 -> 概率论 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 17: 线性代数 -> 离散数学 -> 高等数学 -> 概率论 -> 数据结构 -> 算法设计 -> 计算机网络 -> 操作系统
方案 18: 线性代数 -> 离散数学 -> 高等数学 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 19: 离散数学 -> 高等数学 -> 线性代数 -> 概率论 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 20: 离散数学 -> 高等数学 -> 线性代数 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 21: 离散数学 -> 高等数学 -> 线性代数 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
方案 22: 离散数学 -> 线性代数 -> 高等数学 -> 概率论 -> 数据结构 -> 算法设计 -> 操作系统 -> 计算机网络
方案 23: 离散数学 -> 线性代数 -> 高等数学 -> 概率论 -> 数据结构 -> 算法设计 -> 计算机网络 -> 操作系统
方案 24: 离散数学 -> 线性代数 -> 高等数学 -> 概率论 -> 数据结构 -> 操作系统 -> 算法设计 -> 计算机网络
```

8 开发问题与解决

在系统开发过程中, 我遇到了多个技术挑战和实际问题。

首先遇到的是拓扑序列数量可能过多的问题。理论上, 一个具有 n 个节点的 DAG, 其拓扑序列的数量可能是指数级的。为了解决这个问题, 我引入了 MAX_ORDER 常量, 限制系统最多存储 1000 个拓扑序列。当序列数量达到这个限制时, DFS 搜索会提前终止。在实际应用中, 1000 个序列已经足够为用户提供全面的参考。

其次, 在实现 DFS 回溯算法时, 状态恢复是一个容易出错的地方。最初的设计中, 我在选择课程后减少了其后继课程的入度, 但在回溯时只是简单地将所有课程的入度恢复为初始值。这种方法在简单的依赖关系中可能有效, 但在复杂的依赖关系中会导致错误。通过引入 change 数组, 记录每次选择实际修改了哪些

课程的入度，然后在回溯时只恢复这些课程的状态，解决了这个问题。这个改进不仅提高了正确性，也提高了效率，因为只需要恢复实际修改的状态，而不是所有状态。

性能优化也是一个持续的过程。最初的 DFS 实现中，每次递归调用时都会扫描所有课程来寻找候选课程，这在课程数量较多时会导致效率低下。通过优化，我在递归调用之间传递必要的信息，减少了不必要的扫描，提高了运行效率。

9. 总结与体会

通过本系统的设计与实现，我深刻认识到离散数学理论在实际问题解决中的重要性。拓扑排序作为一个经典的图算法，不仅在理论上严格的数学基础，在实际应用中也具有广泛的价值。将理论知识转化为可运行的程序，不仅需要理解算法的原理，还需要考虑实现的细节、效率、健壮性等多个方面。

系统的开发过程让我更加熟悉了图的数据结构和算法实现。邻接表作为一种高效存储稀疏图的数据结构，在实际编程中需要考虑很多细节，如边的添加、遍历、删除等操作。深度优先搜索回溯算法虽然概念上简单，但实现时需要仔细管理状态，确保递归的正确性和效率。这些实践让我对图算法的理解从理论层面提升到了实现层面。

此外，系统的测试和调试过程让我认识到了全面测试的重要性。通过设计各种测试用例，包括正常情况、边界情况和异常情况，能够发现并修复许多潜在的问题。测试不仅是验证系统正确性的手段，也是理解系统行为的重要途径。

这个系统还有进一步扩展的可能性。例如，可以增加课程的学分信息，支持按学期划分课程的功能；可以增加课程的难度评估，帮助学生合理安排学习负担。这些扩展方向为后续的研究和开发提供了思路。

总之，本系统的设计与实现是一次理论与实践相结合的成功尝试。通过这个项目，我不仅掌握了拓扑排序算法的原理和实现，也提高了解决实际问题的能力，为未来的学习和研究奠定了坚实的基础。

10 部分源代码

10.1 图的邻接表构建

//=====图的邻接表=====

```
int head[NMAX];    //存该节点最后一条加入的边的序号
int to[EMAX];      //存边指向的顶点
int nextEdge[EMAX]; //这条边的下一条、同起点节点的边
int edgeCount = 0;  //已添加的边计数
// 添加一条从 u 指向 v 的有向边
void add_edge(int u, int v) {
    if (edgeCount >= EMAX) {
        cout << "错误：边数超过 EMAX 限制\n";
        exit(1);
    }
    to[edgeCount] = v;
    nextEdge[edgeCount] = head[u];
```



```

    head[u] = edgeCount;
    edgeCount++;
}

```

10.2 kahn 算法检测是否存在环

//=====kahn 算法检测是否有环=====

```

bool has_cycle(int n) {
    int tmp[NMAX];
    for (int i = 0; i < n; i++) {
        tmp[i] = indegree[i];
    }

    int count = 0;
    bool visited[NMAX] = { false };

    while (true) {
        int found = -1;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && tmp[i] == 0) { //没有被访问过并且入度为 0，可以取出
                found = i;
                break;
            }
        }
        if (found == -1) //没有满足条件的顶点
            break;
        visited[found] = true;
        count++;
        for (int e = head[found]; e != -1; e = nextEdge[e]) {
            int v = to[e];
            tmp[v]--;
        }
    }
    return count != n; //没有取出所有点说明存在环
}

```

10.3 回溯算法生成所有拓扑排序

```

void dfs_all(int depth) {
    if (totalTopo >= MAX_ORDER) return; //如果排序数量超过上限，直接截断
    // 如果当前 depth == courseCount，表示已选满一条完整序列，记录
    if (depth == courseCount) {
        if (totalTopo < MAX_ORDER) { //加入检测
            for (int i = 0; i < courseCount; i++) {
                allTopo[totalTopo][i] = path[i];
            }
        }
    }
}

```

```

        totalTopo++;
    }
    return;
}
int candidate[NMAX];    //收集入度为 0 且未被使用的节点
int candidateCount = 0;    //候选的节点数
for (int i = 0; i < courseCount; i++) {
    if (!used[i] && degCopy[i] == 0) {
        candidate[candidateCount++] = i;
    }
}
if (candidateCount == 0) //没有候选节点，直接返回
    return;
//对每个候选依次选择递归
for (int k = 0; k < candidateCount; k++) {
    int u = candidate[k];
    //选择 u，写入 path
    used[u] = true;
    path[depth] = u;
    //选择 u 之后，u 的后续课程不受 u 约束，入度需要修改
    int change[NMAX];    //存放改变的节点
    int changeCount = 0;
    for (int e = head[u]; e != -1; e = nextEdge[e]) { //遍历所有 u 的后继节点
        int v = to[e];
        degCopy[v]--;
        change[changeCount++] = v;
    }
    dfs_all(depth + 1);    //递归下一个深度
    //撤销选择 u，恢复入度
    used[u] = false;
    for (int t = 0; t < changeCount; t++) {
        int v = change[t];
        degCopy[v]++;
    } //path 在下一次选择时会被覆盖，不必额外清除
}
}
}

```

参考文献

- [1] 朱军军. 拓扑排序算法对排课方案判定的应用[J]. 电脑知识与技术, 2010, (16): 4587-4589.
- [2] 殷人昆主编. 数据结构 用面向对象方法与 C++语言描述 第3版[M]. 北京: 清华大学出版社, 2021.
- [3] (美) 肯尼思·H. 罗森 (KENNETH H. ROSEN) 著, 徐六通, 杨娟, 等. 离散数学及其应用[M]. 北京: 机械工业出版社, 2022.
- [4] 薛春艳. 基于邻接表结构的拓扑排序的全序列算法研究[J]. 现代计算机, 2016, (19): 74-76.